contrastBERT: Behavioral Anomaly Detection for Malware using Contrastive Learning

John Carter[1 [0000−0003−4391−0269]], Spiros Mancoridis[1 [0000−0001−6354−4281]], Pavlos Protopapas[2 [0000−0002−8178−8463]], and Brian Mitchell[1 [0009−0002−7820−3927]]

[1] Drexel University, Philadelphia, PA, USA
{jmc683,mancors,bmitchell}@drexel.edu
[2] Harvard University, Cambridge, MA, USA
pavlos@seas.harvard.edu

Abstract. Behavioral anomaly detection is a useful tool for finding malware attacks. Detection is possible using binary classification, where the behavior of benign software and malware is considered during the detector learning process. However, such classifiers are less likely to detect zero-day attacks, as the set of possible malware is enormous and changing. An alternative to binary classification is anomaly detection, which considers only the behavior of benign software to train a malware detector. Recent advances in Large Language Models (LLMs) in combination with Contrastive Learning have created a unique opportunity to develop anomaly detectors to expose malware threats. Our detector, **contrastBERT**, defines its language vocabulary as the set of calls to the operating system and the sequences of such calls as sentences. These sequences of system calls are logged during the execution of both benign software on the system to be monitored as well as separately during malware execution. **contrastBERT** is evaluated using a similarity distribution between benign sample sentences fed to an isolation forest (IF), which provides an effective separation between benign and malicious behavior when trained on embeddings from **contrastBERT**. Our results show that using this approach, several types of common malware behavior patterns can be detected with a high detection rate and a low false positive rate.

Keywords: contrastive learning · BERT · behavioral anomaly detection · malware detection

1 Introduction

Behavioral anomaly detection is a rapidly developing and changing domain that lies at the intersection of machine learning and cybersecurity. Although malware detection is a very active area of research, much of it relies on classification models, which in this space are not as useful because of the many malware variants and obfuscation techniques that malicious actors employ. Modern malware can obfuscate their behavior sufficiently to appear different from known malware

stored in an anti-virus database. Because of this, a promising way to detect malware in this ever-changing problem space is anomaly detection, where normal behavior can be modeled and no known malware is needed to train a detector. This type of anomaly detection also makes the task for malicious actors more difficult because they need to know how each system is designed and how it behaves in order to make their mimicry malware appear benign.

Just a few years ago, researchers in this space were using simple natural language processing (NLP) techniques for data transformations, such as bag-of-word n-grams and term frequency-inverse document frequency (TF-IDF). Since then, transformer models have revolutionized not only behavioral anomaly detection but also the artificial intelligence community and society as a whole. The effects of this phenomenon have been seen in everything from deep-fake videos to AI assistants writing code using tools like Cursor and CoPilot. These recent advances were influenced by a seminal paper entitled "Attention is All You Need" by Vaswani et al., a paper that extended the work of Bahdanau et al., which eliminates the need for recurrent and convolutional models [2] [22].

The vanilla Bidirection Encoder Representations from Transformers (BERT) model was published the following year, which introduced the novel concept of training with context from both sides, rather than in a left-to-right fashion as with other models at the time [8]. Although there have been many advances in transformers in the past six years, the BERT models, which are maintained by Hugging Face [23], remain useful for research in custom model training, with several variations of BERT including RoBERTa, DistilBERT, and ALBERT.

In this work, we train a custom BERT model using contrastive learning. The BERT framework provides the embedding layer and the encoder. The input data to this model are kernel-level operating system calls collected on a Linux router that serves an Internet of Things (IoT) ecosystem. This ecosystem consists of multiple devices and multiple communication protocols, which makes it realistic for evaluation. System calls provide a representation for the activities that happen on a device because every process running on the device uses them to ask the kernel for the resources needed to execute their task. The system calls act as a proxy for system behavior that does not require any modification to the systems themselves, and include familiar tasks such as `read`, `write`, and `close`.

The system calls are collected during times of strictly benign behavior, which are used for training the model (and separately for evaluation), as well as periods of malicious behavior, in which we filter the data to collect only system calls run by processes that are started by the malware. This is necessary to ensure that only malicious behavior is recorded in the malicious data. In general, malware exhibit common behaviors that appear in a wide range of malware. In this work, we call these common behaviors **malware patterns**, which we later define and use. Additionally, we create a stealthy Advanced Persistent Threat (APT) malware for evaluation. These are explained in detail in Section 3.

Each system call is embedded into 64-length vector representations using a custom-trained model we call **sys2vec**, introduced in prior work, which is a model trained from scratch using the Word2vec model framework provided by a

package called Gensim [4] [5] [18]. This model provides vector embeddings that allow system calls to be semantically grouped by functionality, such as grouping file I/O operations, network socket operations, and more. The **sys2vec** preprocessing step provides additional context for the model before being passed into the BERT embedding and encoder layers.

The vectors are then separated into anchors, positive examples, and negative examples to be used in a custom triplet loss function based on the cosine similarity metric between vector representations outputted from the BERT encoder. One of the most important aspects of training a contrastive model with triplet loss is the selection of the triplets, which we explain in detail in Section 3.

After the **contrastBERT** model is trained, the pipeline output is evaluated using a simple (single variable) isolation forest available from scikit-learn [16]. The isolation forest is fit on benign data unseen during training, which allows us to determine the generalization of this method more effectively than by splitting one dataset into training and testing sets. After fitting the model with benign data, we predict whether eight different unknown datasets, collected during times of malware execution, are anomalous or not. The results for each of these malware patterns are shown in Table 2.

The experimental results indicate that not only is the model able to distinguish benign behavior from malicious behavior, but it is able to do so without prior knowledge of any flavor of malware, since only benign data are used in the training of the model. This result is useful in combating modern malware, such as zero-day attacks, since no prior knowledge is necessary to detect a malware infection with high efficacy. The experimental results are discussed further in Section 4.

The paper is organized as follows. First, we discuss work related to behavioral malware detection, contrastive learning, and LLMs for anomaly detection. Next, the data collection apparatus is described in detail, including the router and connected devices, as well as the eBPF sensor that lives on the router and records system calls. Malware patterns are then described, explaining their behavior and relevance to this research topic. Next, we describe how we transform a system call into a vector representation using **sys2vec**, perform triplet mining on a benign dataset of such vectors for contrastive learning, and feed the resulting triplets of data into our **contrastBERT** model to learn the benign behavioral profile of the router. Lastly, we evaluate the entire pipeline using an isolation forest model and describe the results for each malware pattern, and then describe future work.

2 Related Work

2.1 Behavioral Malware Detection

Traditionally, malware detection was conducted by signature matching, which compared suspicious binaries to known malware binaries stored in anti-virus company databases. This method has fallen out of favor as the primary detection

method because malware have evolved to become more stealthy. Two types of malware have led the way: metamorphic malware, which changes its code on each replication, and polymorphic malware, which encrypts its executable files [11] [15]. These two types of malware make systems vulnerable to zero-day attacks, as it is no longer possible to scan their binary files and compare them with known malware binaries [15].

As this has become the case, behavioral malware detection has become an increasingly preferred method of malware detection. Behavioral malware focuses on creating a behavioral profile of a system in a benign state to compare against potentially infected states, rather than relying on checking the code of the malware. This method is more robust to unknown threats, which makes it more effective against malware that is previously unseen or malware that has obfuscated its binary code or its behavior.

2.2 Contrastive Learning

The contrastive learning technique used to train our BERT model was first introduced in the work of Schroff et al., which uses triplets of data in the training process [21]. The triplets consist of two similar samples, called an anchor and a positive example, and a dissimilar example, called a negative example. The goal of the training process is to separate each pair of positive examples from negative examples using a distance margin [21].

This algorithm was originally developed to help with facial recognition software developed by Google, but has since been used for other applications. These include object tracking [9], as well as several types of anomaly detection, including detecting video anomalies [3] and driving anomalies using a conditional generative adversarial network (CGAN) [17].

There are many ways of setting up the loss function using triplet loss, a few of which are implemented for use in PyTorch, such as TripletMarginLoss and TripletMarginWithDistanceLoss [6] [7]. In this work, we chose to train using a custom loss function based on the cosine similarities of the anchor and the positive example and the anchor and the negative example because we found that using the cosine similarity between the vectors for loss calculation between examples yielded superior results. The custom loss function is defined in Section 3.

2.3 Large Language Models for Anomaly Detection

Anomaly detection problems can be characterized as "finding a needle in a haystack," and because of this, the introduction of the Attention mechanism and the resulting transformer models have largely left traditional ML models behind due to their capacity for huge amounts of context, which aids in better performance.

The most visible LLMs today are generative pre-trained transformers (GPT), like OpenAI's ChatGPT and Google's Gemini. Some work has been done on the use of GPTs in the context of anomaly detection, such as the work by Ali et al.,

which used GPTs to help create HuntGPT, an intrusion detection dashboard built on a random forest (RF) model [1]. In this example, the GPT model is used to create a conversational agent to convey any detected threats in an easily digestible format [1].

A different study addressed the use of an LLM to detect semantic anomalies, which the authors described as system-level failures in semantic reasoning within autonomous systems, such as the autopilot features of a Tesla [10]. Their proposal to counter these semantic reasoning failures was to insert an LLM to act as a monitor within the autonomous system to identify any anomalies it encounters [10].

There are also works related to malware classification using LLM, such as those by Zhao et al., which focuses on detecting Android malware using an OpenAI model [24]. This differs significantly from our work, since we are interested in anomaly detection rather than classification, which is more useful for zero-day attacks, and we used a custom-trained model that is built specifically for our system call language.

3 Technical Approach

3.1 Data Collection

The data used in this work consist of kernel-level system call data from a Linux router connected to multiple IoT devices residing in an ecosystem, which we call IoTOwl. An observability platform we created, called **SkyShark**, is used to collect the system calls. **SkyShark** uses the Linux subsystem **eBPF** for the efficient logging of all system calls that occur on the device [14].

eBPF allows user programs to be executed in the kernel safely. These programs can register interest in targeted kernel services, such as system calls that enter or exit the kernel, and stream real-time data with high performance and granularity. **SkyShark** provides several customization parameters for the type of data to collect in addition to the raw system call numbers, such as timestamps and filtering by a subset of process IDs (PIDs). In this case, we chose to log all system calls along with their calling process IDs for all processes running on the device, since we do not know ahead of time which processes might be infected by malware.

The devices connected to the ecosystem use three types of common IoT communication protocols and include the following:

- PurpleAir PA-II Air Quality Sensor (WiFi)
- BerryMed Pulse Oximeter (BLE)
- Google Home (WiFi)
- Philips Hue Light Bulb (Zigbee)
- A phone and laptop (WiFi)

Each of the IoT devices, including the air quality sensor, the pulse oximeter, and the light bulb, sends its current sensor data to the Linux router, which acts

Fig. 1. Example IoTOwl server readings from the AWS server, including from the air quality sensor, the Zigbee light bulb, and the pulse oximeter.

as a gateway to a server that resides on an Amazon Web Services (AWS) `EC2` instance using Prometheus and Grafana. All of the malware execution and system call collection happen on the router, while the AWS server simply dashboards and visualizes the sensor readings from the IoT devices. This includes current temperature and humidity readings, blood oxygen and pulse readings, and current brightness, hue, and saturation of the Philips bulb. Sending the current sensor readings to the AWS server allows for a more realistic environment in which to collect the system call data.

An example of how the server might look in operation is shown in Figure 1, which is a screenshot of the server residing in AWS. The screenshot shows a real-time data display from the PA-II air quality sensor, the Philips Hue bulb, and the pulse oximeter sensor.

3.2 Malware

Malware Patterns Several types of malware are used to evaluate this work, including a collection of malware behaviors that we denote as **malware patterns**. These patterns are designed to emulate the singular behaviors of malware from the time they land on a device to the time they execute. For this work, we identified the following malware patterns. Each of these is implemented to be simple and lightweight, both for simplicity purposes and for generalizability.

The first behavior is **download**, which downloads code from a `git` repository and removes it. This emulates malware behavior after the malware lands on the device and attempts to connect to a server to download its malicious code.

The next behavior is **traverse**, which navigates the file system and runs a `stat` on each file it encounters. This emulates a surveilling malware that is looking for sensitive files that contain passwords, financial data, and other types of private data to exploit.

Another behavior is **encrypt**, which traverses the file system and encrypts each file using the `gpg` command. This malware emulates many types of malware that look at files and then encrypt them to make them unusable to the file owner, most notably in ransomware, which has been an increasingly expensive problem over the last decade.

Next is the **rename** behavior, which traverses the file system and changes the name of each file and then changes it back. Though this seems to be a trivial malware behavior, it emulates how a malware might find a file, and then rename it to put it in another location to hide it from its owner.

Another behavior is **compile**, which compiles C code into executable files and subsequently removes them. This emulates many types of malicious code that need to be compiled to run, as running C code is much more lightweight than running code of an interpreted language, like Python, and thus makes it less noticeable.

The **combo** malware seeks to combine several of the most common patterns into a comprehensive malware type by first downloading code from the Internet, compiling the code, and then traversing the file tree and encrypting / decrypting each file it encounters.

The last core malware behavior we explored is **lateral**, which attempts to look into sensitive password files, such as `/etc/shadow`, and then uses `ping` to get active IP addresses on the subnet. Subsequently, it tries to `ssh` into each of them to perform a lateral movement on the network.

Advanced Persistent Threat In addition to these general behaviors, we created an Advanced Persistent Threat (APT), which is designed to be a stealthy malware that slowly exfiltrates data without being detected. As noted above, it is written in C to be more lightweight and less detectable. It is implemented using the standard client-server architecture, in which a server waits for a connection from a client, and as soon as the client connects, it starts sending files over for exfiltration.

The **APT** provides a bandwidth parameter, which allows the user to exfiltrate data very slowly and makes detection more difficult. A bandwidth limit of 100 kilobytes per second was chosen to strike a balance between the amount of data exfiltrated being useful and not being too stealthy. It also provides sleep time and run time parameters that allow users to specify how long the **APT** should run and how long it should be dormant between exfiltration times.

3.3 Data Preprocessing using sys2vec

Before passing the system call data into the model for training, it is first passed through a custom trained Word2vec model, called **sys2vec**, which was introduced in previous work [4] [5]. Although the BERT model used during training

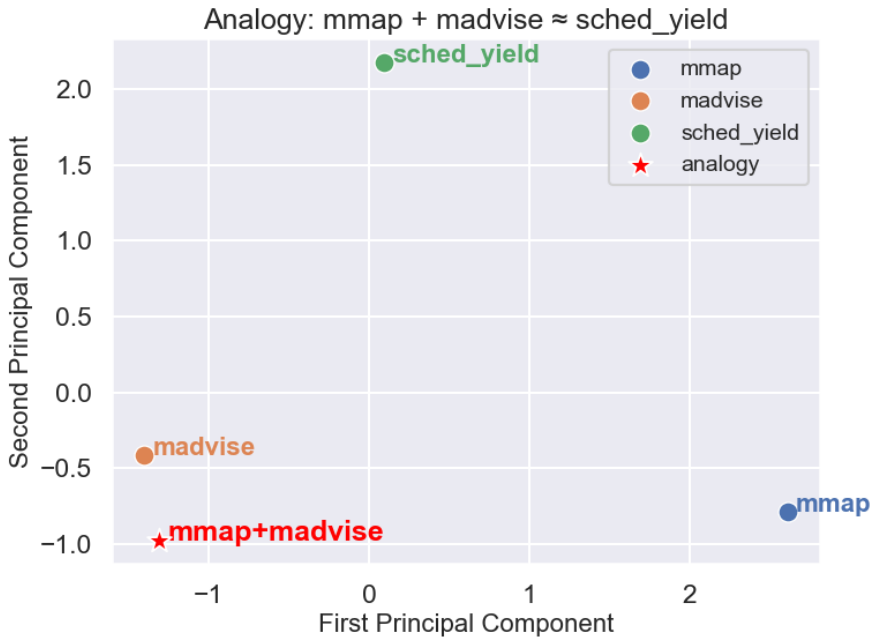

Fig. 2. This example shows how **sys2vec** is able to capture context between semantically-related system calls. The dimensionality has been reduced from 64 to 2 using PCA.

has its own embedding layer, passing the **sys2vec** vectors to the model initially enables the model to start with even more context, thus improving results. The **sys2vec** model was trained for 100 epochs and used a sentence length of 500 contiguous system calls, which was found to be optimal for computing efficiency and contextual sufficiency. The data used for training **sys2vec** comprise several datasets that contain only benign data from the device. 486,345 sentences were used for training, and the embedding length of each vector is 64. The final vocabulary size is 164, since we only observed 164 system calls during our extensive benign data collection processes.

To demonstrate that the `sys2vec` model can learn the relationships between system calls, we provide two informative figures. The original Word2vec paper by Mikolov et al. provides a classic analogy to depict how normal algebraic expressions can be performed on the vector representations of words [13]. In their case, a pertinent example is King-Man+Woman=Queen, while in our case using system calls, a pertinent example is `mmap` + `madvise` ≈ `sched_yield`. This example is reasonable since, after mapping memory, processes might call `madvise` using the `MADV_DONTNEED` option, which can cause the OS kernel to yield control to another scheduled process waiting for service. Figure 2 depicts the example

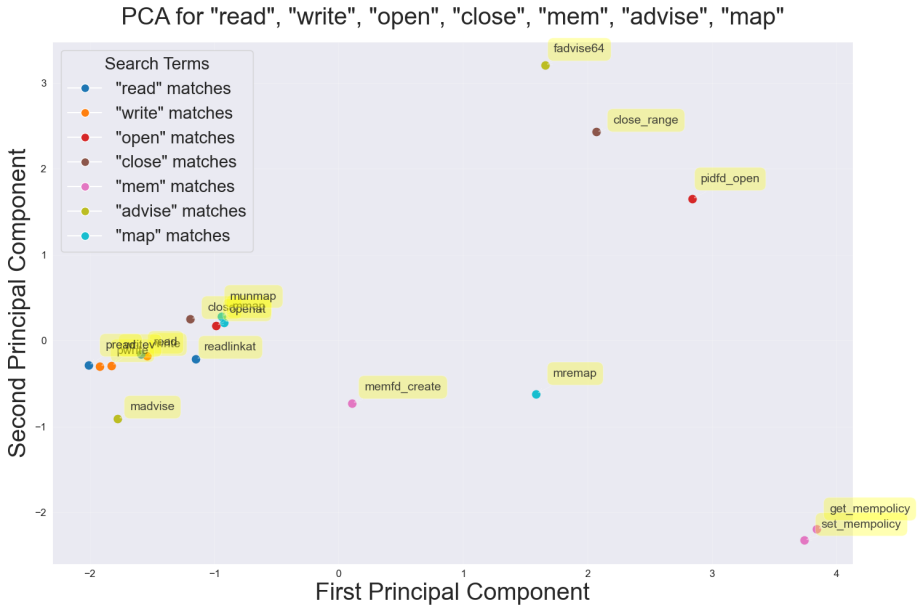

Fig. 3. Example **sys2vec** embeddings reduced from 64 dimensions to 2 using PCA. This shows how **sys2vec** is capable of clustering system calls semantically by showing examples of common system calls like `read`, `write`, `open`, and `close`.

using PCA dimensionality reduction, where `sched_yield` is the closest vector in the vocabulary to the `mmap` + `madvise` expression.

Figure 3 provides an example of **sys2vec** embeddings for a variety of related and unrelated system calls. This figure depicts the strengths and weaknesses of **sys2vec**'s ability to cluster functionally similar system calls, such as `write` and `writev`, as well as semantically close system calls like `openat`, `read`, `write`, and `close`. As shown in the figure, the model performs well in grouping many of them, such as the `read`- and `write`-related system calls, but does not do as well with `madvise` and `fadvise64`, for example.

3.4 Triplet Selection

We swept over a range of parameters to select the best type of triplet data to train the model, as the quality of the triplet data can make all the difference when training a contrastive learning model.

After selecting the anchor, which is each sentence in the data, and the positive example, which we chose to be the next sequential sentence in the data, the negative sample was constructed using a random sample that has not yet been seen as we loop through the data. In other words, the random negative sample is a sentence between the current positive sample and the total number of sentence vectors. For example, if the anchor is S_0, the positive example is S_1, the negative

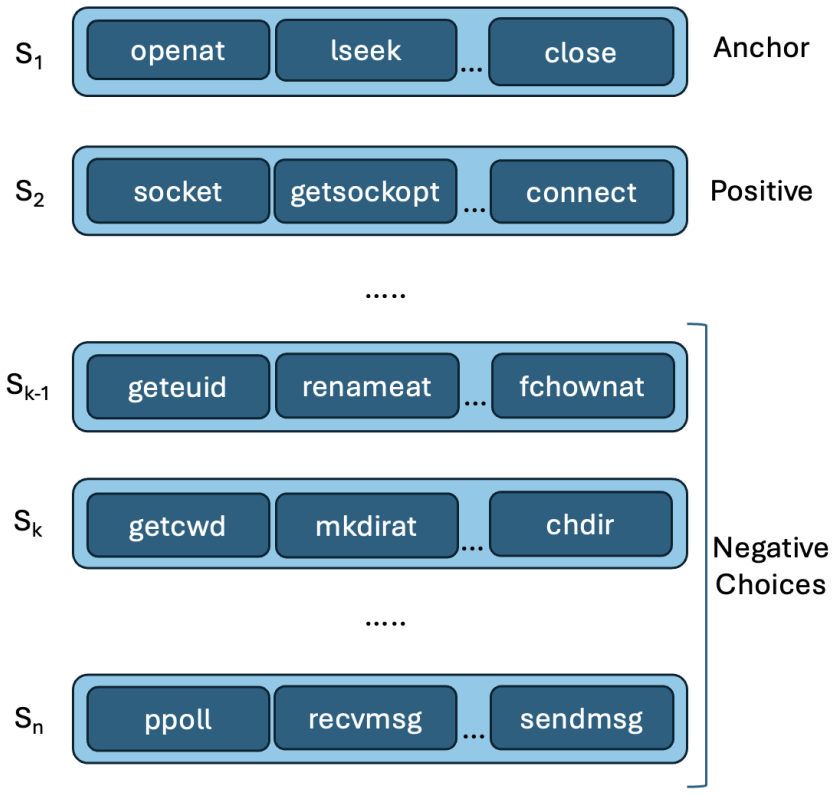

Fig. 4. Triplet selection comprises an anchor, a positive, and a negative sentence, where the negative sentence is randomly selected from unseen vectors. In this example, sentences between S_{k-1} and S_n are choices to select from randomly.

sample is chosen from sentences in the range S_2 to S_n. Figure 4 shows this triplet relationship in more detail.

3.5 Triplet Loss

The loss function of the model is designed to separate positive and negative examples and is defined as follows, where $\cos(\mathbf{a}, \mathbf{b})$ denotes the cosine similarity between two vectors:

$$\mathcal{L} = \mathbb{E}\left[\max\left(0,\ -\cos(\mathbf{a}, \mathbf{p}) + \cos(\mathbf{a}, \mathbf{n}) + \text{margin}\right)\right] \tag{1}$$

During the experimentation process, we also tried using the PyTorch function TripletMarginLoss using Euclidean distance as the distance metric between the anchor, positive, and negative examples. This provided useful results, but we

found that the use of cosine similarity between vectors generated superior results. The triplet loss formula, and the steps leading to it, is shown in Figure 5.

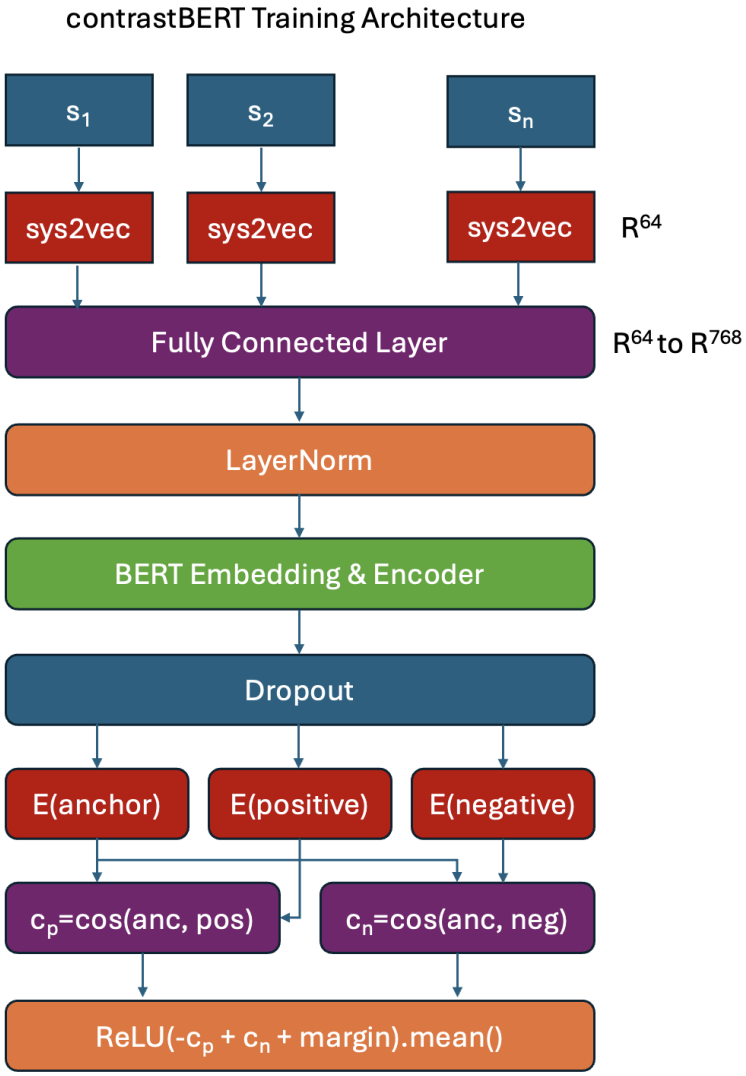

Fig. 5. contrastBERT architecture - the lifecycle of a system call from individual system call, like **open**, to passing it through our custom triplet loss function.

3.6 contrastBERT Training

The architecture of **contrastBERT** is shown in Figure 5, which describes the life cycle of a single system call observed through the custom loss function.

The first layer shows an observed system call, which is subsequently passed through the **sys2vec** layer, where it is transformed into a 64-length vector. This static embedding provides an initial context for the BERT model during the training process. As part of our ablation study, we also trained a separate BERT model using random 64-length embeddings to show the relative difference in performance with and without using **sys2vec**. We normalized the random vectors to reduce the influence of the magnitude of the vectors, though we did try non-normalized random vectors for completeness and found it yielded worse results overall. The random vectors are fixed for all of the data used in this work. Next, the data are passed through a fully connected layer to expand their dimensionality from 64 to BERT's standard dimensionality of 768. The data are then passed through a LayerNorm layer, before being passed to the BERT Embeddings() layer and the BERT Encoder. The BERT Embeddings() layer provides additional dynamic embedding context that builds on what is provided by the **sys2vec** embeddings.

Once the data are passed through the encoder and a Dropout layer, the data are pooled using mean pooling and then the anchor, positive, and negative example sentences are all normalized. The cosine similarity is then taken between the anchor and the positive example and between the anchor and the negative example. These two cosine similarities are the basis of our custom loss function, which attempts to draw the positive example closer to the anchor while simultaneously pushing the negative example away.

Table 1. Number of observations for each dataset. Note that the number of observations for the benign system calls are truncated to 1M observations, while the non-benign dataset totals are shown after PID-filtering, meaning the total system calls observed correspond directly to malware execution.

Dataset	**# Observations**
benign_180min	1,000,000
benign2_180min	1,000,000
benign2_60min	1,000,000
Download	3,262
Traverse	793,115
Encrypt	140,935
Rename	29,641,915
Compile	779
Lateral	177,033
Combo	28,027
APT	55,592

3.7 contrastBERT Evaluation

For evaluation, we test using three separate benign datasets in addition to the malware dataset being evaluated to assess the generalizability between similar, yet distinct, benign datasets. Although the ecosystem configurations are identical for each of the benign data collections, there can sometimes be noise in the data from, for example, kernel activity or Domain Name System (DNS) updates.

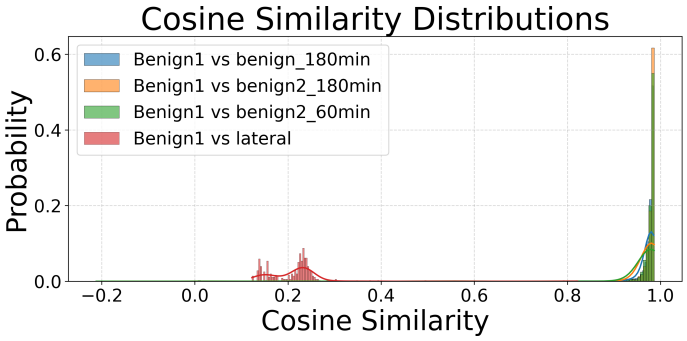

Fig. 6. Cosine similarities or the **lateral** malware pattern. The figure shows how the **lateral** cosine similarities are mostly separable from the benign datasets.

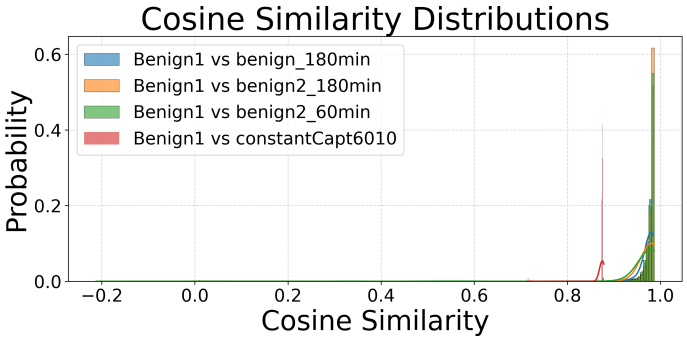

Fig. 7. Cosine similarities for the **APT** malware. The figure shows how the **APT** is very close to the benign datasets, and there is some benign extending to the left side of the graph.

The use of several benign datasets during evaluation ensures that the model is not learning one distinct dataset but rather a more general benign behavioral profile. In total, four datasets are used in the evaluation of each malware pattern: one benign, denoted $benign_1$, a second, separate benign, denoted $benign_2$,

a third, separate benign, denoted $benign_3$ and an unknown dataset, denoted *unknown*. The number of observations for these datasets, as well as for each of the malware datasets, is shown in Table 1.

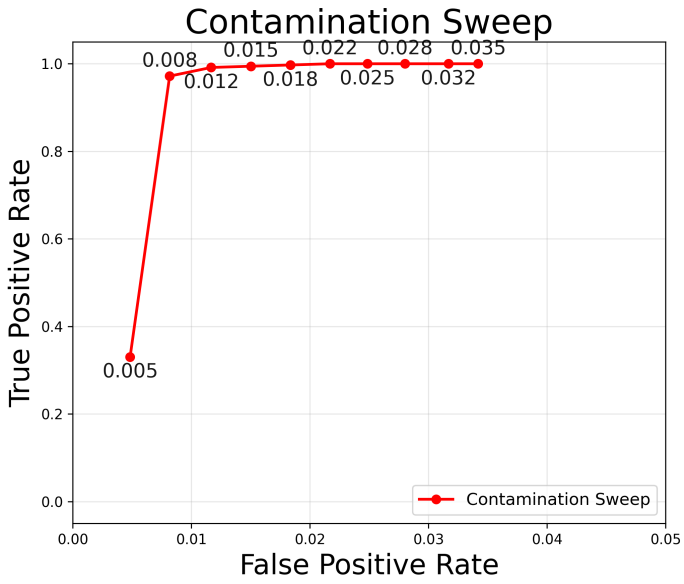

Fig. 8. Contamination sweep for **lateral** malware. Note that the x-axis scale is only from 0.00 to 0.05 to provide a zoomed-in view. The figure shows the isolation forest efficacy at several contamination rates. Using a contamination rate of 0.025 results in 100% detection and minimizes the false positives as much as possible while maintaining that detection rate, while using a lower contamination rate, such as 0.005, results in a better false positive rate but only about 33% true positive rate.

Similar to the model training phase, we compare the usefulness of the **sys2vec** embeddings as part of our ablation study during the evaluation phase and pass the raw system call datasets through both the **sys2vec** model to generate the 64-length embeddings as well as using the fixed normalized random embeddings as input to **contrastBERT**. Both types of embeddings are then passed separately through the **contrastBERT** model to obtain the 768-length embedding vector from the BERT encoder.

Pairwise cosine similarities are taken for $benign_1$ and itself, $benign_1$ and $benign_2$, $benign_1$ and $benign_3$, and $benign_1$ versus *unknown*. After computing pairwise cosine similarities, we take the mean of each of the pairwise cosine similarities between datasets. Examples of the resulting mean of pairwise cosine similarities are depicted in Figures 6 and 7. Figure 6 for the lateral malware shows cosine similarities that are very separable from benign, while Figure 7 for the APT malware shows cosine similarities that are less separable from the

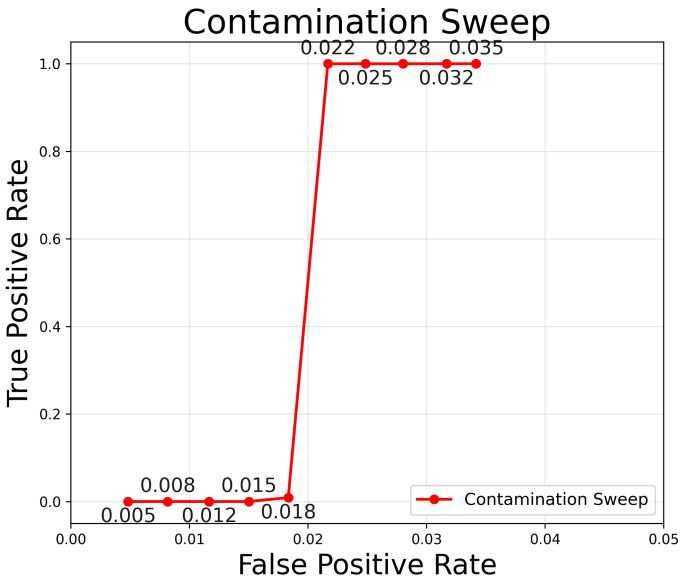

Fig. 9. Contamination sweep for **APT** malware. Note again that the x-axis scale is only from 0.00 to 0.05 to provide a zoomed-in view. For this malware, the true positive rate stays much closer to 0 until a 0.022 contamination rate is used. This means that there is much less of a tradeoff here, because without that high of a false positive rate, the detector is unusable.

three benign similarities. The mean of each of the pairwise cosine similarities between the three benign datasets is used to fit an isolation forest. We also tried using the embeddings themselves to fit the isolation forest directly as part of our ablation study, but this generally yielded slightly worse results. Figure 10 shows the average malware detection at several isolation forest contamination with and without cosine similarities as well as using **sys2vec** embeddings versus fixed random embeddings.

The isolation forest is an off-the-shelf model from scikit-learn, which uses all default parameters except for the contamination rate, which is the expected ratio of anomalies in the training dataset. We swept over a number of rates to find the optimal parameter for our problem space, as depicted in Figures 8 and 9. These figures show the tradeoff between detection rate and false positive rate for the **lateral** and **APT** malware. In the next section, we explore three contamination rates in detail, which bring into focus the tradeoff between detection and false positives.

4 Experimental Results

Table 2 shows the results for all types of malware from the isolation forest using both **sys2vec** and fixed random embeddings and three contamination rates:

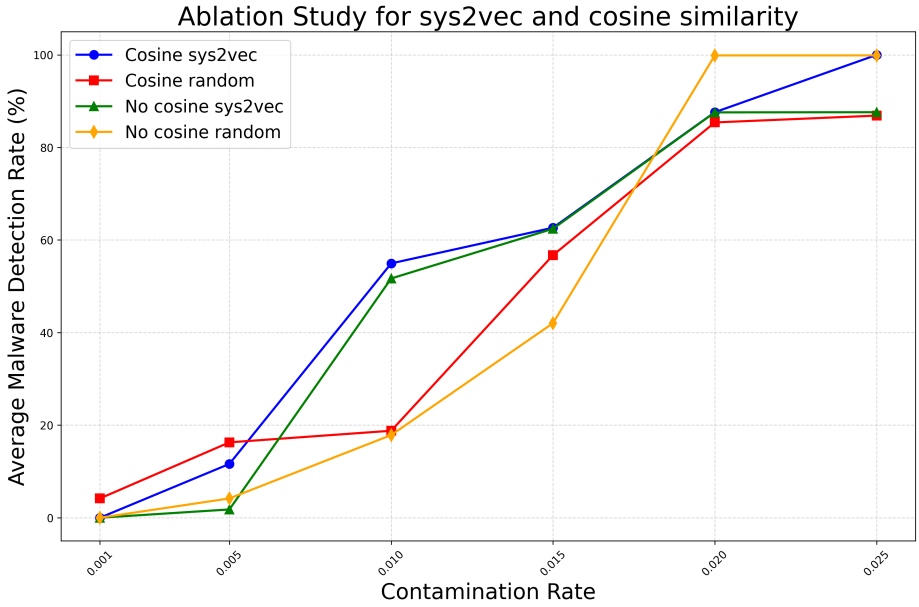

Fig. 10. An ablation study evaluating the impact of type of embedding (**sys2vec** vs. fixed random) and isolation forest input (mean of pairwise cosine similarities vs. raw BERT embeddings) on the average of malware detection across malware patterns as a function of the contamination rate. For FPR ≤ 2, **sys2vec** using cosine similarities generally performs best and is thus the method examined in detail in Table 2. For higher false positive rates, there is less dependence on using **sys2vec** and cosine similarities.

0.005, 0.015, and 0.025. These contamination rates were chosen to illustrate the tradeoff between detection rate and false positive rate referred to in Figures 8 and 9.

Overall, the results show that using a contamination rate of 0.025 results in high detection efficacy of 100% across all malware patterns using **sys2vec** embeddings and almost 100% for random embeddings, but with it an average false positive rate across the benign datasets of 2.5%. Similarly, using a 0.015 contamination rate yields usable detection for five of the eight malware patterns with a lower 1.50% false positive rate using **sys2vec** embeddings, and four malware patterns using fixed random embeddings. This contamination rate might be preferred for a use case in which a low false positive rate is more important than the best detection possible. Lastly, the 0.005 contamination rate yields essentially unusable results for all malware patterns examined, save perhaps for the **traverse** malware pattern using fixed random embeddings.

As shown in Table 2, the false positive rate can be lowered for some malware patterns using a lower contamination rate, such as in Figure 8, but the detection rate falls accordingly. However, for some malware, such as the **APT** shown in Figure 9, a higher contamination rate is necessary for detection.

Table 2. Detection rates of malware samples at varying contamination levels, defined as the percentage of samples flagged as anomalies. We show both **sys2vec** input vectors as well as fixed random input vectors to compare the relative effectiveness of both using our **contrastBERT** model. These results all use cosine similarity in the evaluation.

Malware	0.005		0.015		0.025	
	sys2vec	**random**	**sys2vec**	**random**	**sys2vec**	**random**
Download	0.00%	50%	100%	66.67%	100%	83.33%
Traverse	0.00%	80.26%	0.06%	100%	100%	100%
Encrypt	11.64%	0.00%	100%	99.27%	100%	100%
Rename	48.27%	0.00%	100%	2.40%	100%	100%
Compile	0.00%	0.00%	100%	0.00%	100%	100%
Lateral	33.05%	0.00%	99.44%	99.72%	100%	100%
Combo	0.00%	0.00%	1.79%	85.71%	100%	100%
APT	0.00%	0.00%	0.00%	0.00%	100%	11.71%

5 Conclusion

In this work, we collected system call data from a Linux router serving a real IoT ecosystem, which was then passed through an ML pipeline consisting of our custom **sys2vec** embedding model and a custom-trained anomaly detection model we call **contrastBERT**. The anomaly detection model is trained using contrastive learning, which uses triplets of data composed of an anchor, positive and negative example. For each sentence in the training data, the first sentence is the anchor, the second is the positive example, designed to be most similar to the anchor, and the third is the negative example, which is selected randomly from the unseen sentences at the time of processing each anchor.

After training **contrastBERT**, we passed new testing data through **sys2vec** and **contrastBERT** and used the resulting embeddings to compute pairwise cosine similarities between each of the four evaluation datasets and $benign_1$, and took the mean of the pairwise cosine similarities. These similarities were used to fit an isolation forest using varying contamination rates to illustrate the tradeoff between detection rate and false positive rate. These values are shown in Table 2.

We also conducted an ablation study to determine how useful the **sys2vec** embeddings were to the overall pipeline by training a separate **contrastBERT** model on fixed random vectors in place of the **sys2vec** vectors and using the same vectors for evaluation of the model. Figure 10 illustrates the general performance tradeoff between the two and Table 2 provides more detail for each type of malware. Generally, it was found that the **sys2vec** embedding pipeline slightly outperformed the fixed random embedding pipeline.

The **sys2vec** and **contrastBERT** pipeline was able to detect malware perfectly at the 0.025 contamination rate, although with that comes a false positive rate of about 2.5%, which may be high for some problem domains. Because of

this, we also tried using a 0.015 contamination rate. This yields excellent results for about half of the malware patterns examined, and a lower false positive rate of about 1.5%. A more false positive-conscious problem domain may prefer this contamination rate, since it still yields usable results for many malware patterns. Lastly, we found that a 0.005 contamination rate with a 0.5% false positive rate yielded nearly unusable results for most malware.

The contribution of this research work is a highly effective ML pipeline that uses **sys2vec** and **contrastBERT** to detect a variety of common patterns of malware behavior running on a router in a realistic IoT ecosystem. We custom-designed the **sys2vec** embedding framework as well as the **contrastBERT** model using contrastive learning specifically to be used in a system call language space. The use of anomaly detection for malware detection rather than classification provides a more robust solution since no prior malware knowledge is needed to train **contrastBERT**, making it especially suitable for zero-day malware.

5.1 Future Work

One area we would like to explore in future work is to provide more options to process the embeddings outputted by the BERT encoder. An option that we have begun exploring is ColBERT, which is a late-interaction model that provides multiple vector representations for each token [20]. There are also other flavors of BERT models to try, such as RoBERTa [12] and DistilBERT [19], which were briefly explored in previous work by the authors. In addition, it would be useful to automate the selection of a contamination rate so that a grid search is unnecessary.

Another area of future work is to expand the data collection to augment the system call data with network traffic data. Although the system calls provide enough behavioral context to construct a useful model, we think the results could be improved with additional network context, such as open ports, BLE device detection, and more.

Lastly, we would like to do more in relation to peripheral device infection. In this study, the router serving a realistic ecosystem was infected, but in many cases it may be an edge device that is infected, such as the air quality sensor or the phone in this ecosystem. It is useful to study the feasibility of detecting all malware on any edge device using only data from the router, since the router sits as the gatekeeper to the rest of the devices on the network it serves and can keep the devices sitting behind the router secure without requiring a separate detection appliance on each device.

Acknowledgments. This research was funded by the Auerbach Berger Chair of Cybersecurity held by Spiros Mancoridis.

References

1. Ali, T., Kostakos, P.: Huntgpt: Integrating machine learning-based anomaly detection and explainable ai with large language models (llms) (2023), https://arxiv.org/abs/2309.16021
2. Bahdanau, D., Cho, K., Bengio, Y.: Neural machine translation by jointly learning to align and translate (2016), https://arxiv.org/abs/1409.0473
3. Biradar, K.M., Mandal, M., Dube, S., Vipparthi, S.K., Tyagi, D.K.: Triplet-set feature proximity learning for video anomaly detection. Image and Vision Computing **150**, 105205 (2024). https://doi.org/https://doi.org/10.1016/j.imavis.2024.105205, https://www.sciencedirect.com/science/article/pii/S026288562400310X
4. Carter, J., Mancoridis, S., Protopapas, P.: sysbert: Improved behavioral malware detection using bert trained on sys2vec embeddings. In: Proceedings of the 57th Hawaii International Conference on System Sciences. pp. 7122–7131 (January 2025). https://doi.org/10.24251/HICSS.2025.851
5. Carter, J., Mancoridis, S., Protopapas, P., Galinkin, E.: Behavioral malware detection using a language model classifier trained on sys2vec embeddings. In: Proceedings of the 56th Hawaii International Conference on System Sciences. pp. 7582–7591 (January 2024). https://doi.org/10.24251/HICSS.2024.911
6. Contributors, P.: torch.nn.tripletmarginloss. https://pytorch.org/docs/stable/generated/torch.nn.TripletMarginLoss.html, accessed: 2025-04-14
7. Contributors, P.: torch.nn.tripletmarginwithdistanceloss. https://pytorch.org/docs/stable/generated/torch.nn.TripletMarginWithDistanceLoss.html, accessed: 2025-04-14
8. Devlin, J., Chang, M., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. CoRR **abs/1810.04805** (2018), http://arxiv.org/abs/1810.04805
9. Dong, X., Shen, J.: Triplet loss in siamese network for object tracking. In: Computer Vision – ECCV 2018: 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part XIII. p. 472–488. Springer-Verlag, Berlin, Heidelberg (2018). https://doi.org/10.1007/978-3-030-01261-8_28, https://doi.org/10.1007/978-3-030-01261-8_28
10. Elhafsi, A., Sinha, R., Agia, C., Shah, A., Vorobeychik, Y., Sanders, W.H.: Semantic anomaly detection with large language models. Autonomous Robots **47**, 1035–1055 (2023). https://doi.org/10.1007/s10514-023-10132-6, https://doi.org/10.1007/s10514-023-10132-6
11. Kale, A.S., Pandya, V., Di Troia, F., Stamp, M.: Malware classification with word2vec, hmm2vec, bert, and elmo. Journal of Computer Virology and Hacking Techniques **19**(1), 1–16 (2023). https://doi.org/10.1007/s11416-022-00424-3
12. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., Stoyanov, V.: Roberta: A robustly optimized bert pretraining approach (2019), https://arxiv.org/abs/1907.11692
13. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space (2013), https://arxiv.org/abs/1301.3781
14. Mitchell, B.S., Chandnani, A., Carter, J., Roumelioti, D., Mancoridis, S.: Malware detection in cloud native environments. In: Proceedings of the 2024 Artificial Intelligence and Cloud Computing Conference (AICCC 2024). Tokyo, Japan (Dec 2024)
15. Natsos, D., Symeonidis, A.L.: Transformer-based malware detection using process resource utilization metrics. Results in Engineering **25**, 104250 (2025). https://

doi.org/https://doi.org/10.1016/j.rineng.2025.104250, https://www.sciencedirect.com/science/article/pii/S2590123025003366
16. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
17. Qiu, Y., Misu, T., Busso, C.: Use of triplet-loss function to improve driving anomaly detection using conditional generative adversarial network. In: 2020 IEEE 23rd International Conference on Intelligent Transportation Systems (ITSC). pp. 1–7 (2020). https://doi.org/10.1109/ITSC45102.2020.9294562
18. Rehurek, R., Sojka, P.: Gensim–python framework for vector space modelling. NLP Centre, Faculty of Informatics, Masaryk University, Brno, Czech Republic **3**(2) (2011)
19. Sanh, V., Debut, L., Chaumond, J., Wolf, T.: Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter (2020), https://arxiv.org/abs/1910.01108
20. Santhanam, K., Khattab, O., Saad-Falcon, J., Potts, C., Zaharia, M.: Colbertv2: Effective and efficient retrieval via lightweight late interaction (2022), https://arxiv.org/abs/2112.01488
21. Schroff, F., Kalenichenko, D., Philbin, J.: Facenet: A unified embedding for face recognition and clustering. In: 2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 815–823 (2015). https://doi.org/10.1109/CVPR.2015.7298682
22. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention is all you need. In: Proceedings of the 31st International Conference on Neural Information Processing Systems. p. 6000–6010. NIPS'17, Curran Associates Inc., Red Hook, NY, USA (2017)
23. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Brew, J.: Huggingface's transformers: State-of-the-art natural language processing. CoRR **abs/1910.03771** (2019), http://arxiv.org/abs/1910.03771
24. Zhao, W., Wu, J., Meng, Z.: Apppoet: Large language model based android malware detection via multi-view prompt engineering (2024), https://arxiv.org/abs/2404.18816